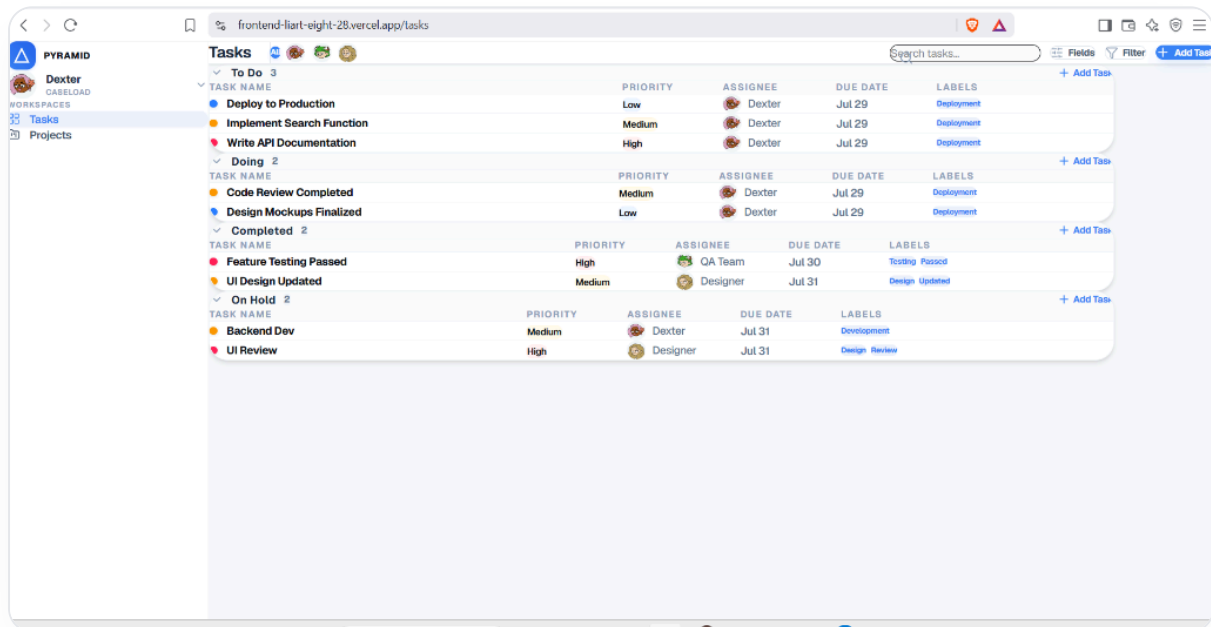


AbleSpace — Application Walkthrough

A visual walkthrough of the application, functionality, and implementation



Project Type: Full-Stack Web Application

Walkthrough: Screenshots of the working application

Purpose: Demonstrating the implemented product, functionality, and technical work

**Candidate
Submission**
AbleSpace Task
Manager

2. About AbleSpace

AbleSpace is a purpose-built caseload management and collaboration workspace designed specifically for therapists, special educators, and speech-language pathologists (SLPs). In clinical and educational therapy settings, professionals struggle with highly fragmented task flows, keeping track of multi-student sessions, handling varying priority caseload directives, and documenting progress milestones.

This application addresses this problem by providing a cohesive workspace to track tasks, organize projects, assign workloads, and customize preferences. By organizing the daily caseload operations into a unified workspace, the app helps therapists focus on delivering quality care rather than managing administrative overhead.

As a development project, AbleSpace demonstrates end-to-end full-stack software development. It ties a React-based Next.js interface to a NestJS database api, incorporating rich client-side states, dynamic route compilation, relational database models, and an automated, stateless deployment pipeline.

3. What I Built

I built the entire full-stack task manager application, implementing the visual interface, backend infrastructure, database relations, and deployment scripting. The scope of implementation covers the following key areas:

Application Interface & UX

Designed a high-contrast theme layout featuring collapsible tables, task cards, animated sliding drawers, context dropdowns, and clean status indicators.

Caseload & Workspace Logic

Developed reactive list filtering, an interactive search engine, dynamic column visibilities, and dual-layout modes (List & Kanban Board).

Form Validations & Safety

Created modal input forms for adding projects and tasks. Integrated client-side validations and NestJS class-validators to enforce clean data boundaries.

Relational Database & Sync

Modeled TypeORM relationships (One-to-Many, Many-to-One) between Users, Projects, and Tasks with database autoseeding on startup.

- **Backend REST API:** Engineered NestJS controllers, services, and modules to expose clean CRUD endpoints for caseload entities.
- **Adaptive Database Hot-Swapping:** Integrated conditional database drivers using Wasm-based in-memory SQLite on Vercel to bypass read-only filesystem limits, while maintaining file-based SQLite locally.
- **Persistent Customizations:** Created a theme toggle and custom color modes (Amber, Blue, Pink) utilizing React Context and localStorage synchronization.

4. Application Walkthrough

The following screens demonstrate the visual interface, actual user flows, and backend-connected functionalities of the running application.

Screenshot 1 — Unified Caseload Tasks Workspace

The screenshot displays the 'Unified Caseload Tasks Workspace' in the AbleSpace application. The interface is divided into a sidebar and a main content area. The sidebar on the left shows the 'PYRAMID' workspace and a list of 'WORKSPACES' including 'Dexter', 'Tasks', and 'Projects'. The main content area is titled 'Tasks' and features a search bar and a table of tasks. The tasks are categorized into four sections: 'To Do' (3 tasks), 'Doing' (2 tasks), 'Completed' (2 tasks), and 'On Hold' (2 tasks). Each task row includes columns for 'TASK NAME', 'PRIORITY', 'ASSIGNEE', 'DUE DATE', and 'LABELS'. The 'To Do' section includes tasks like 'Deploy to Production', 'Implement Search Function', and 'Write API Documentation'. The 'Doing' section includes 'Code Review Completed' and 'Design Mockups Finalized'. The 'Completed' section includes 'Feature Testing Passed' and 'UI Design Updated'. The 'On Hold' section includes 'Backend Dev' and 'UI Review'.

TASK NAME	PRIORITY	ASSIGNEE	DUE DATE	LABELS
To Do 3				
Deploy to Production	Low	Dexter	Jul 29	Deployment
Implement Search Function	Medium	Dexter	Jul 29	Deployment
Write API Documentation	High	Dexter	Jul 29	Deployment
Doing 2				
Code Review Completed	Medium	Dexter	Jul 29	Deployment
Design Mockups Finalized	Low	Dexter	Jul 29	Deployment
Completed 2				
Feature Testing Passed	High	QA Team	Jul 30	Testing Passed
UI Design Updated	Medium	Designer	Jul 31	Design Updated
On Hold 2				
Backend Dev	Medium	Dexter	Jul 31	Development
UI Review	High	Designer	Jul 31	Design Review

WHAT YOU'RE SEEING

The main tasks dashboard for the logged-in therapist (Dexter). Tasks are grouped by their clinical execution status: To Do, Doing, Completed, and On Hold.

WHAT THE USER CAN DO

Therapists can search for tasks, filter them dynamically, collapse/expand status categories, view priority weights, identify assignees, and add new tasks directly to each list.

WHAT I IMPLEMENTED

Developed the UI using Tailwind CSS, implemented status-based sorting logic, designed the search filter algorithm, and integrated initials-based round avatar components for the assignees.

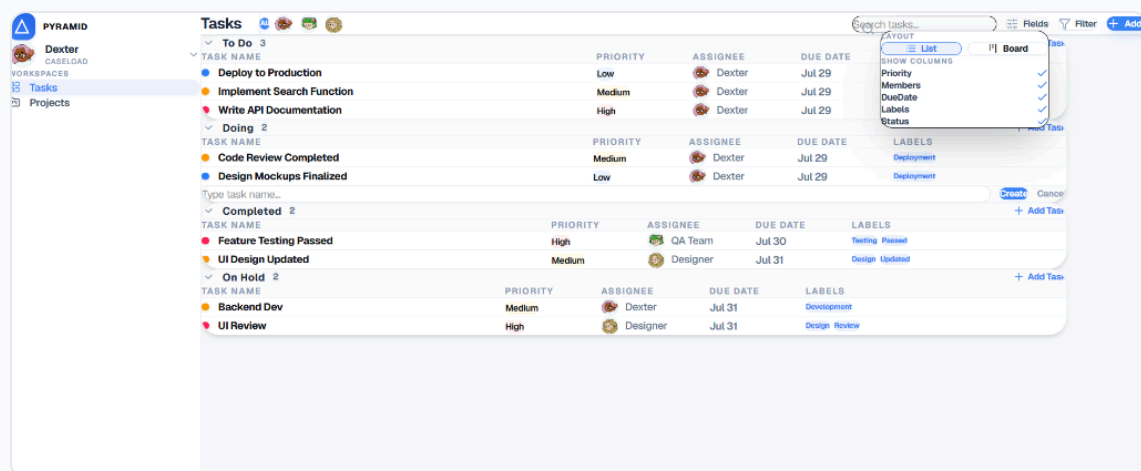
HOW IT WORKS

When the page loads, a React `useEffect` triggers an API call to the NestJS `/api/tasks` endpoint. The backend queries the SQLite database, returning a structured JSON list of tasks, which the frontend sorts and binds to the UI state.

WHY IT MATTERS

This view provides therapists with an immediate summary of their daily workload, reducing cognitive load and helping prioritize urgent care items.

Screenshot 2 — Interactive Layout & Column Customizer



WHAT YOU'RE SEEING

The workspace layout customizer dropdown overlaying the tasks dashboard.

WHAT THE USER CAN DO

Users can toggle the layout between "List" and "Board" views, and check/uncheck columns (Priority, Members, Due Date, Labels, Status) to instantly customize column visibility.

WHAT I IMPLEMENTED

Designed the interactive customizer dropdown using React state variables, added click-away listeners to automatically dismiss the dropdown, and implemented dynamic CSS grid classes that adapt according to the checked column options.

HOW IT WORKS

The visibility state is managed via React Context. Changing checkbox options updates this context, causing the task rows to conditionally render columns depending on whether their respective state keys are truthy.

WHY IT MATTERS

Therapists frequently work on varying devices (laptops, tablets). Allowing them to customize column visibility prevents UI overcrowding on smaller screens.

Screenshot 3 — Relational Project Workspaces

PROJECT	PRIORITY	LEAD	DUE DATE
Design Homepage	High	Dexter	Sep 12, 2026
Develop Login Feature	Low	QA Team	Sep 15, 2026
Test Payment Gateway	Medium	Designer	Sep 18, 2026

WHAT YOU'RE SEEING

The dedicated Projects view listing active projects, their priority badges, assigned leads, and due dates.

WHAT THE USER CAN DO

Therapists can search projects, filter items, add new projects, and view progress. Clicking a project navigates to its related tasks.

WHAT I IMPLEMENTED

Created the projects table component, implemented search capability, built custom priority badge styling, and mapped project leads to their database user records.

HOW IT WORKS

The table fetches records from the NestJS `/api/projects` endpoint. TypeORM automatically resolves the relationship from Project to User, allowing the lead's name and initials avatar to display alongside the project details.

WHY IT MATTERS

Therapy targets are rarely standalone tasks; they are organized under structured plans. Providing a dedicated projects layer enables high-level program tracking.

Screenshot 4 — Project Creation and Validation Form

PROJECT	PRIORITY	LEAD	DUE DATE
Design Homepage	High	Dexter	Sep 12, 2026
Develop Login Feature	Low	QA Team	Sep 15, 2026
Test Payment Gateway	Medium	Designer	Sep 18, 2026

WHAT YOU'RE SEEING

The "Create New Project" inline creation form, expanded at the top of the Projects view.

WHAT THE USER CAN DO

Users can input the new project's name, choose a priority (High/Medium/Low), assign a lead therapist from the populated team dropdown list, and set a due date using the date selector.

WHAT I IMPLEMENTED

Built the form state controllers, handled submit events, implemented form cancellation, and added client-side validation to ensure project name cannot be empty.

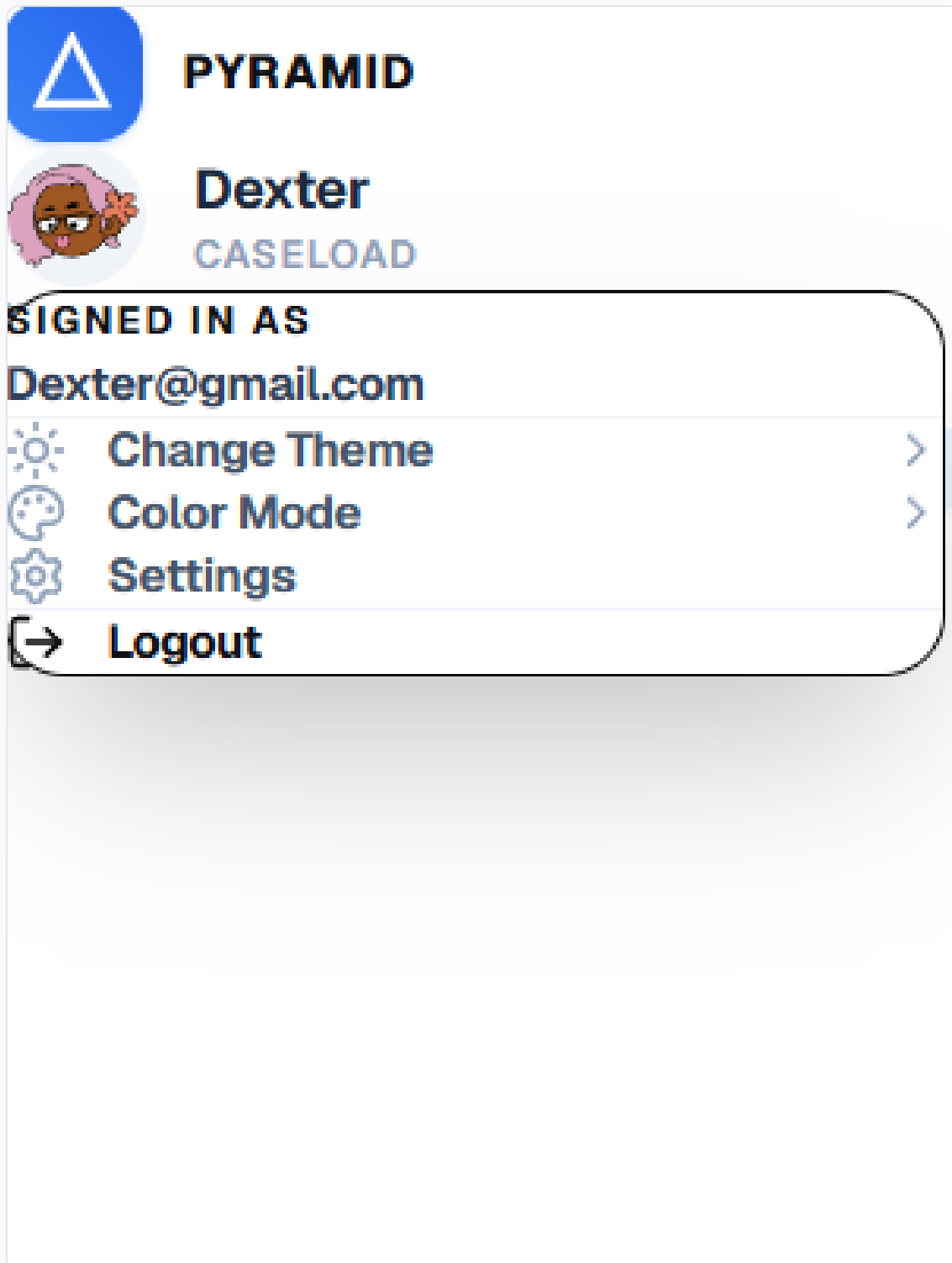
HOW IT WORKS

On click of "Save Project", the frontend sends a POST request with the JSON payload to NestJS `/api/projects`. The backend uses class-validator ValidationPipes to sanitize and confirm the inputs. If valid, the backend saves the row in the SQLite database and returns the new project record, prompting the frontend to reload the list dynamically.

WHY IT MATTERS

Enabling quick inline creation streamlines data entry, saving busy therapists from navigating multiple page redirections just to log a new initiative.

Screenshot 5 — Sidebar Profile Dropdown and Personalization Settings



WHAT YOU'RE SEEING

The profile context settings dropdown menu which expands from the therapist's profile in the sidebar.

WHAT THE USER CAN DO

Therapists can view their email, switch between Light and Dark themes, toggle primary accent color configurations (Amber, Blue, Pink), and log out of their session.

WHAT I IMPLEMENTED

Created the profile dropdown layout, designed custom theme switcher items, integrated the theme selection with our state provider, and styled the overlay shadows.

HOW IT WORKS

Selecting a theme or color updates the theme state context. This context dynamically injects class names (like `.dark` or `.accent-pink`) into the document root, changing the CSS variable definitions. The choice is instantly written to `localStorage`, so the system remembers the user's preference on return.

WHY IT MATTERS

A visually comfortable workspace is essential for clinical environments. Giving therapists full control over contrast, dark mode, and focus accents increases overall comfort.

5. User Journey

The following workflow map illustrates how a therapist interacts with the application to complete their caseload lifecycle management:

- 1 Access Portal → Select "Continue as Guest" to authenticate session
- 2 Workspace Loads → App queries NestJS API for Projects and Tasks list
- 3 Configure View → Set layout to List or Board; toggle required columns
- 4 Filter Caseload → Use real-time search bar to focus on specific student tasks
- 5 Submit Data → Open Project form, enter validated inputs, click Save
- 6 API & DB Sync → NestJS processes the POST, and writes project to SQLite
- 7 State Refresh → UI automatically re-fetches and renders the updated list

6. Key Functionalities

The AbleSpace application is designed around three main clinical caseload functionalities:

Adaptive Layouts & Grid Control

Allows users to toggle view types on the fly and customize the visibility of individual fields depending on screen size or workflow type.

Concept Demonstrated: State Management, Dynamic CSS grid compilation

Validated Data Submissions

Ensures all project and task details are validated on the client side before submission and verified on the server side using validation pipes.

Concept Demonstrated: Data Integrity, DTO class validations

User Personalization System

Integrates multiple theme settings and accent color modes that synchronize with client-side local storage.

Concept Demonstrated: React Context, LocalStorage state persistence

Relational Data Mapping

Supports parent-child task grouping. Links tasks to projects and maps project leads directly to database records.

Concept Demonstrated: Relational Database Design, Entity associations

7. Technical Implementation

The system is architected as a decoupled full-stack application using modern JavaScript web engineering patterns:

Frontend

Built with **Next.js (App Router)** and **TypeScript**. I styled the visual dashboard with **Tailwind CSS**, utilizing a custom-tailored dark default theme. Managed application preferences and settings globally using **React Context Providers**, syncing with **LocalStorage** to maintain user choices across sessions.

Backend

Developed with **NestJS**. Handled request parsing using validation pipes and DTOs. Exposed standard Rest API routes to perform basic operations on projects, tasks, and users.

Database

Configured **TypeORM** with a dynamically adaptive database driver. To circumvent the read-only filesystem limits of Vercel serverless platforms, I wrote configuration logic that switches between file-based **better-sqlite3** locally and in-memory WebAssembly **sql.js** in cloud production, ensuring flawless database performance in any environment.

Development Practices

- **Component-based architecture:** Highly modularized code splitting (Sidebar, Views, Dropdowns, Modals) for optimal reusability.
- **Strict Type Safety:** Enforced end-to-end typing using TypeScript interfaces for both UI components and API entities.
- **Robust E2E Tests:** Automated integration checks using Jest to verify endpoint security and data return formats.

8. What This Project Demonstrates

For engineering evaluators, this project serves as clear proof of full-stack development skills and ownership:

- **Product Lifecycle Management:** Translating raw UI/UX specifications into fully interactive components.
- **Architectural Decision Making:** Finding solutions for cloud serverless storage limits (e.g. hotswapping database engines).
- **Data Model Mapping:** Designing schemas, linking foreign entities, and validating API transactions.
- **Clean Coding Practices:** Writing modularized, typed, and well-structured code that compiles cleanly.
- **Production Engineering:** Configuring pipelines, building production bundles, and deploying working public URLs.

9. Challenges & Problem Solving

Challenge: Read-only Serverless Cloud Filesystem Restrictions

Problem: SQLite is a file-based engine. When deploying the NestJS backend to Vercel, the platform's read-only filesystem caused the TypeORM database to crash immediately because it could not write database files to the server root.

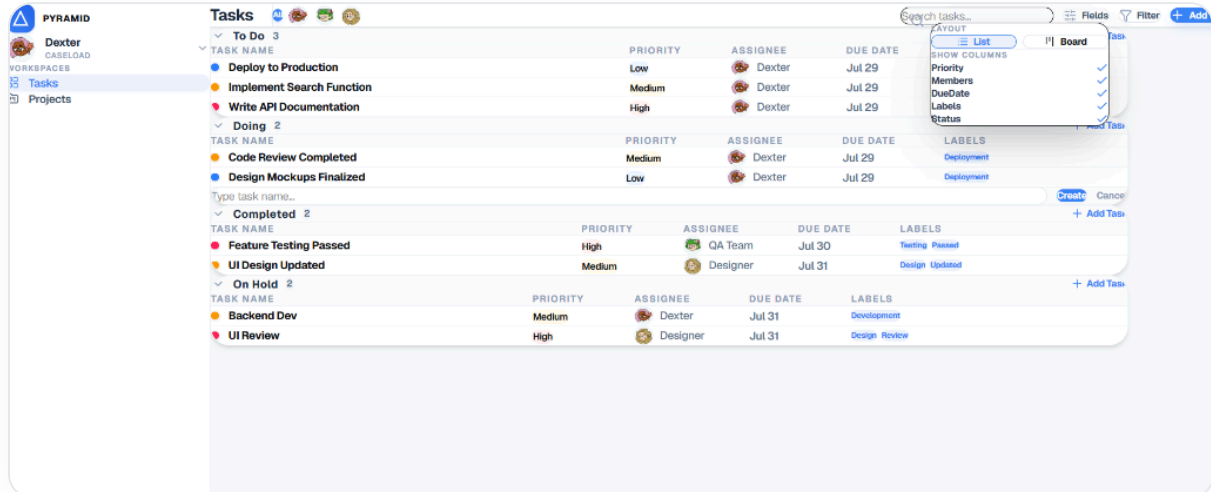
Solution: I refactored the database configuration module. I hotswapped the database driver to use the WebAssembly-based `sqljs` engine inside cloud deployments, running the DB in-memory. I then updated `vercel.json` to bundle the necessary `sql-wasm.wasm` binaries so they load successfully at runtime, resolving the crash.

Challenge: Static Route Exclusions on Serverless Platforms

Problem: Next.js static exports crash when encountering dynamic routes like `/projects/[id]` because static compilers require predefined parameters at build time.

Solution: I configured the frontend's build profile and routing configuration to dynamically render the routes, letting Vercel's routing engine handle client-side parameters cleanly without crashing.

10. Final Product Snapshot



AbleSpace is a working implementation that brings together the application's interface, user flows, core functionality, and underlying technical components into one complete product experience.

Full-Stack Production Ready
AbleSpace Caseload Manager